

UCS503 — SOFTWARE ENGINEERING

TeamSync

A Cross-College Collaboration Platform for Hackathon Team Formation and Project Collaboration

Mid-Semester Evaluation

Aadita Garg · 1024030461
Ridhima Goel · 1024030466
Vedika Gupta · 1024030437

Group 3C32 · BE Third Year, CSED
Submitted to Prof. Asif
Computer Science & Engineering Department
Thapar Institute of Technology, Patiala

The Problem

Finding the right teammate across colleges is still an unsolved, informal process.

1 Fragmentation

Teammate search happens through scattered contacts, WhatsApp/Discord groups, or social posts — no shared, searchable channel.

2 No Cross-College Visibility

A student needing a React developer and a student who has that exact skill elsewhere have no platform to find each other.

3 Low Trust / Verification

Open forums can't confirm a profile genuinely belongs to a verified student, or that claimed skills reflect real work.

4 Poor Skill Discoverability

No structured way to filter people by tech stack, GitHub activity, availability, or complementary skills.

Project Objectives

SMART objectives — each scoped to a concrete feature, a numeric target, and a week in the 12-week plan.

1

Verified Platform

Cross-college platform for student profiles, tech-stack showcases, and listings — deployable to staging by end of Iteration 1 (Week 7).

2

Skill Discovery

Discover teammates via complementary skills, interests, and verifiable GitHub activity — measured by profile completeness & GitHub linkage rate.

3

Ranked Matching

Heuristic ranked matching for project/hackathon requirements — target median time-to-first-match of ≤ 24 hours during pilot.

4

Pilot Validation

Validate via a multi-college pilot (15–25 users, 2+ colleges) — report cross-college match rate & connection-to-team conversion.

System Architecture

A typical 3-tier web architecture.



Core Workflow

- 1 Student registers with verified college email & builds a skill profile, linking GitHub repos as proof of work
- 2 Student posts a requirement ("looking for teammates") or browses/searches existing listings & profiles
- 3 Matching engine surfaces ranked candidates based on complementary skills and verified activity
- 4 Both parties send/accept a connection request, unlocking in-app chat
- 5 If a team forms, members get a shared workspace for the hackathon/project duration

Technical Stack

Framework / Language

React.js + Tailwind CSS (frontend)
Node.js + Express.js (backend API)

Database

MongoDB Atlas — stores users, listings, connections, and messages

Tools

GitHub Actions for CI/CD — build, test, lint, deploy to staging

Libraries

GitHub REST API client for repository/profile linkage
Real-time messaging library for in-app chat

Implementation Details

Module 1 — Core Functionality

- Verified sign-up restricted to valid college email domains
- Structured skill profile: tech stack, domain interests, experience level, GitHub-linked portfolio
- Listing creation and full CRUD
- Basic browse/search over listings and profiles
- Connection requests & in-app chat, unlocked on acceptance
- Timestamps logged at each state transition to support the time-to-first-match metric

Module 2 — Secondary Features

- Complementary-skill matching engine: structured tags normalized (tech stack, domain interest, experience level)
- Weighted-overlap score combines skill complementarity, interest overlap, GitHub activity, experience, and availability
- Heuristic by design — surfaces top-ranked candidates; never auto-assigns teams
- Team workspace: task allocation and status tracking
- Admin dashboard for pilot metrics

Testing Methodology

1 Unit Testing

Backend unit tests cover authentication, profile/listing CRUD, and the matching-score calculation — run automatically in CI on every change.

2 Integration Testing

End-to-end checks of the core workflow (sign-up → listing → match → connection → chat) against a staging deployment.

3 Performance Testing

Response-time and throughput checks on the matching and search endpoints under simulated pilot-scale load.

4 User Testing

Pilot UAT with 15–25 recruited students across 2+ colleges, timed around an upcoming hackathon, with a 1–5 clarity survey.

Performance Metrics

Targets defined in the proposal — "Achieved" values populate once Week 10 pilot UAT and staging measurements complete.

Metric	Target	Achieved
Time-to-first-match (median)	≤ 24 hours	TBD – pilot pending
Connection-to-team conversion	Target to be set	TBD – pilot pending
Cross-college match rate	Target to be set	TBD – pilot pending
Profile completeness / verification rate	Target to be set	TBD – pilot pending
GitHub linkage rate	Target to be set	TBD – pilot pending
Staging/production uptime (pilot hours)	≥ 99%	TBD – pilot pending

At mid-semester: Objectives 1 & 2 have working implementations in progress. Objectives 3 & 4 depend on the matching engine and pilot, scheduled for Weeks 8–10, and are not yet measurable.

Progress Against the 12-Week Plan

Week 7 — Today



Weeks 1–12

Completed / In Progress (Weeks 1–7 · Initial MVP)

- Verified sign-up & skill profile with GitHub linkage
- Listing creation, CRUD, and browse/search
- Objectives 1 & 2 — working implementations in progress

Scheduled Next (Weeks 8–10)

- Complementary-skill matching engine
- Multi-college pilot UAT (Objectives 3 & 4)
- Time-to-first-match & cross-college rate become measurable

Anticipated Challenges & Mitigation

Cold-start (too few users for meaningful early matches) and cross-college onboarding friction — mitigated by scheduling the pilot around a live hackathon and starting with two colleges rather than an open rollout.

Key Findings So Far

1 Framing matching as heuristic weighted-overlap scoring, rather than a research recommender system, kept the first-iteration scope achievable within the course timeline.

2 Verification via college email plus GitHub linkage gives a low-cost trust signal without building a full identity-verification system.

3 Early implementation trade-offs and user feedback will be recorded here as the pilot progresses.

Future Work & Recommendations

- 1 Extend the matching engine with learned weights once sufficient pilot interaction data is available.
- 2 Explore integration with existing hackathon-registration platforms to reduce cold-start friction.
- 3 Add indexed, paginated search to support scale readiness beyond the two-college pilot.
- 4 Integrate additional verification signals (e.g., LinkedIn, resume parsing) alongside GitHub linkage.
- 5 Evaluate real-world deployment considerations — hosting cost, data retention, and moderation — ahead of any post-course rollout.

Results

Outcome metrics defined for the pilot; most become measurable once the Weeks 8–10 pilot runs.

1

Time-to-First-Match

Target: median ≤ 24 hours from listing to first relevant candidate. Status: pilot pending (Weeks 8–10).

2

Cross-College Match Rate

Target: share of matches formed between students at different colleges. Status: pilot pending.

3

Profile & GitHub Verification

Profile completeness and GitHub linkage rate across sign-ups to date. Status: tracked from launch, in progress.

4

Connection-to-Team Conversion

Target: share of accepted connections that become confirmed teams. Status: pilot pending.

Summary

TeamSync makes cross-college teammate discovery as easy as within-college collaboration.

Verified profiles + GitHub-backed skill signals + heuristic ranked matching + a small multi-college pilot, evaluated primarily through time-to-first-match and cross-college match rate.

Thank you

Questions welcome